SUBCONSULTAS, VISTAS Y MODIFICACIÓN

Clase 3 · Diseño de Base de Datos

Ingeniería en Informática · 2.° año · Universidad Católica de Córdoba

Agenda

0. **Recordatorio — qué venimos sabiendo hacer**
1. **Subconsultas anidadas — `IN`, `NOT IN`, `EXISTS`, `NOT EXISTS`**
2. **Subconsultas en otros lugares — escalares, en `FROM`, `WITH` (CTE)**
3. **Vistas — `CREATE VIEW`, actualización, `WITH CHECK OPTION`**
4. **Modificación de datos — `INSERT`, `UPDATE`, `DELETE` + reglas para no romper nada**

Al final vas a poder armar consultas complejas con varios niveles de anidamiento, exponer abstracciones con vistas y modificar datos con seguridad.

De dónde venimos, a dónde vamos

HASTA ACÁ · CONSULTAS PLANAS

Cada consulta era un único `SELECT`: filtros con `WHERE`, combinación con `JOIN`, resúmenes con `GROUP BY` y `HAVING`, conjuntos con `UNION` / `INTERSECT` / `EXCEPT`.

HOY · CONSULTAS DENTRO DE CONSULTAS

Una consulta puede aparecer *dentro* de otra: en el `WHERE`, en el `FROM`, como un valor, o nombrada con `WITH`. Y a las que se repiten mucho, les vamos a poner un alias permanente: una **vista**.

✓ HILO CONDUCTOR

Al final también vamos a modificar datos (`INSERT`, `UPDATE`, `DELETE`) usando subconsultas para decidir qué tocar.

01

Subconsultas anidadas

Una consulta dentro de otra

¿Qué es una subconsulta?

Una `SELECT` que aparece dentro de otra. La interior se ejecuta primero y su resultado se usa en la exterior.

CONSULTA EXTERIOR

```
SELECT apellido, nombre
FROM alumnos
WHERE id_alumno IN (
```

SUBCONSULTA (INTERIOR)

```
SELECT id_alumno
FROM inscripcion_alumno_comision
WHERE id_comision = 12
```

```
);
```

¿PARA QUÉ SIRVEN?

Para chequear **pertenencia** (¿este id está en este otro conjunto?), **existencia** (¿hay al menos una fila relacionada?), **comparación** (¿es mayor que el promedio?), o para **calcular un valor** que se usa en otra parte de la consulta.

IN: pertenencia a un conjunto calculado

```sql
-- Alumnos inscriptos en alguna comisión de asignaturas obligatorias
SELECT DISTINCT a.apellido, a.nombre
FROM alumnos a
WHERE a.id_alumno IN (
    SELECT i.id_alumno
    FROM inscripcion_alumno_comision i
    INNER JOIN comisiones c   ON i.id_comision = c.id_comision
    INNER JOIN asignaturas asg ON c.id_asignatura = asg.id_asignatura
    WHERE asg.es_obligatoria = 1
);
```

CÓMO SE LEE

"Trae alumnos cuyo `id_alumno` esté en el conjunto de IDs que devuelve la subconsulta". `IN` también acepta listas literales: `WHERE id_departamento IN (1, 3, 5)`.

✓ TUPLA CON IN

Se puede comparar varias columnas a la vez: `WHERE (anio, cuatrimestre) IN (SELECT anio, cuatrimestre FROM ...)`.

NOT IN — la trampa del NULL que ya conocemos

```sql
-- "Docentes que nunca dictaron una comisión"
SELECT apellido, nombre
FROM docentes
WHERE id_docente NOT IN (
    SELECT id_docente FROM comisiones
);
```

⚠ SI LA SUBCONSULTA DEVUELVE ALGÚN NULL, EL RESULTADO SE VACÍA

`x NOT IN (a, b, NULL)` equivale a `x <> a AND x <> b AND x <> NULL`, y la última comparación da `UNKNOWN` → la fila no entra al resultado.

✓ DOS FORMAS DE PROTEGERSE

```sql
-- A: filtrar NULL en la subconsulta
WHERE id_docente NOT IN (
  SELECT id_docente FROM comisiones WHERE id_docente IS NOT NULL
)

-- B (preferida): usar NOT EXISTS, que es inmune a NULL
WHERE NOT EXISTS (
  SELECT 1 FROM comisiones c WHERE c.id_docente = docentes.id_docente
)
```

EXISTS y NOT EXISTS

`EXISTS (subconsulta)` devuelve `TRUE` si la subconsulta produce **al menos una fila**, `FALSE` si no produce ninguna. `NOT EXISTS` es la negación.

EXISTS · AL MENOS UNA

```sql
-- Alumnos que tienen al menos
-- una nota aprobada
SELECT a.apellido, a.nombre
FROM alumnos a
WHERE EXISTS (
    SELECT 1
    FROM notas n
    WHERE n.id_alumno = a.id_alumno
      AND n.aprobado = 1
);
```

NOT EXISTS · NINGUNA

```sql
-- Alumnos que NUNCA aprobaron
-- nada (incluye los que no tienen notas)
SELECT a.apellido, a.nombre
FROM alumnos a
WHERE NOT EXISTS (
    SELECT 1
    FROM notas n
    WHERE n.id_alumno = a.id_alumno
      AND n.aprobado = 1
);
```

✓ TIP

El `SELECT 1` dentro de `EXISTS` es convención: el motor solo chequea si *hay filas*, no le importa qué columnas. Da lo mismo `SELECT *` o `SELECT id`.

EXISTS vs IN — ¿cuándo cada una?

Criterio	IN	EXISTS
Relación con la consulta exterior	Independiente (no correlacionada). La subconsulta se calcula una sola vez.	Suele estar correlacionada: usa columnas de la exterior. Se "evalúa" por cada fila exterior.
Cómo se compara	Valor exterior contra *cada valor* del conjunto.	Solo importa si la subconsulta *devuelve filas o no*.
Comportamiento con NULL	`NOT IN` falla silenciosamente con NULL.	`NOT EXISTS` es robusto: NULL no rompe nada.
Cuándo elegir cada uno	Cuando comparás contra un conjunto de valores conocido y sin NULLs.	Cuando la subconsulta depende de la fila exterior, o cuando hay riesgo de NULL.

CORRELACIONADA VS NO CORRELACIONADA

Una subconsulta es **correlacionada** cuando referencia una columna de la consulta exterior (como `n.id_alumno = a.id_alumno` en el slide anterior). Si no la referencia, es **no correlacionada** y se ejecuta una sola vez para toda la consulta.

SOME y ALL — mención

SQL ofrece dos formas adicionales para comparar contra el resultado de una subconsulta. En la práctica casi todo se resuelve con `IN`, `EXISTS` o agregados, pero conviene reconocerlas si las ven en código ajeno.

```sql
-- Docentes con salario mayor al de ALGÚN docente del depto. 3
SELECT apellido FROM docentes
WHERE salario_base > SOME (
    SELECT salario_base FROM docentes WHERE id_departamento = 3
);

-- Docentes con salario mayor al de TODOS los del depto. 3
SELECT apellido FROM docentes
WHERE salario_base > ALL (
    SELECT salario_base FROM docentes WHERE id_departamento = 3
);
```

Equivalencia útil	Sentido
`= SOME`	Equivale a `IN`
`<> ALL`	Equivale a `NOT IN`

En MySQL `ANY` es sinónimo de `SOME`. No vamos a profundizar — usamos `IN` y `EXISTS` como herramientas principales.

02

Subconsultas en otros lugares

Escalares, en `FROM`, y nombradas con `WITH`

Subconsulta escalar — una fila, una columna

Si una subconsulta devuelve **exactamente una fila con una columna**, se puede usar como si fuera un valor: en `SELECT`, `WHERE`, `HAVING` o `SET`.

```sql
-- Para cada asignatura, cuántas comisiones tiene abiertas
SELECT asg.nombre,
       (SELECT COUNT(*)
        FROM comisiones c
        WHERE c.id_asignatura = asg.id_asignatura) AS cant_comisiones
FROM asignaturas asg
ORDER BY cant_comisiones DESC;

-- En WHERE: docentes con salario mayor al promedio general
SELECT apellido, salario_base
FROM docentes
WHERE salario_base > (SELECT AVG(salario_base) FROM docentes);
```

⚠ **OJO**

Si en tiempo de ejecución la subconsulta devuelve **más de una fila**, el motor lanza error. Si devuelve **cero filas**, el resultado es `NULL`.

Subconsulta en FROM — tabla derivada

Cualquier `SELECT` devuelve una relación, así que se puede usar como tabla en otro `FROM`.

```sql
-- Promedio de notas por alumno → quedarse con los que tienen promedio ⩾ 8
SELECT id_alumno, promedio
FROM (
    SELECT id_alumno, AVG(nota) AS promedio
    FROM notas
    WHERE aprobado = 1
    GROUP BY id_alumno
) AS prom_alumnos
WHERE promedio ⩾ 8
```

¿POR QUÉ NO RESOLVERLO CON HAVING?

Se puede. Pero cuando la consulta se complica (varios niveles de agregación, joins con un resumen previo, etc.), las tablas derivadas son más legibles. Y `WITH` aún más.

⚠ **OBLIGATORIO**

La subconsulta del `FROM` necesita un **alias** (acá `prom_alumnos`). MySQL lo exige siempre.

WITH — vistas temporales (CTE)

Una *Common Table Expression* (CTE) le pone nombre a una subconsulta y la deja usable como tabla en el `SELECT` principal. Mejora la legibilidad cuando hay varios pasos.

```sql
-- Asignaturas con promedio de notas superior al promedio general
WITH prom_por_asig AS (
    SELECT c.id_asignatura, AVG(n.nota) AS prom
    FROM notas n
    INNER JOIN examenes e   ON n.id_examen = e.id_examen
    INNER JOIN comisiones c ON e.id_comision = c.id_comision
    GROUP BY c.id_asignatura
)
SELECT asg.nombre, p.prom
FROM prom_por_asig p
INNER JOIN asignaturas asg ON p.id_asignatura = asg.id_asignatura
WHERE p.prom > (SELECT AVG(prom) FROM prom_por_asig);
```

✓ TIP

Una misma consulta puede definir varias CTEs separadas por coma: `WITH a AS (...), b AS (...) SELECT ...`. MySQL las soporta desde 8.0.

Pausa: probalo vos

Consigna: listar las asignaturas que *nunca* tuvieron una comisión abierta en 2025.

VERSIÓN CON NOT IN

```sql
SELECT id_asignatura, nombre
FROM asignaturas
WHERE id_asignatura NOT IN (
    SELECT DISTINCT id_asignatura
    FROM comisiones
    WHERE anio = 2025
      AND id_asignatura IS NOT NULL
);
```

VERSIÓN CON NOT EXISTS

```sql
SELECT asg.id_asignatura, asg.nombre
FROM asignaturas asg
WHERE NOT EXISTS (
    SELECT 1
    FROM comisiones c
    WHERE c.id_asignatura = asg.id_asignatura
      AND c.anio = 2025
);
```

CUÁL PREFERIR

La de la derecha. Es robusta ante NULLs sin necesidad de filtrarlos a mano, y es la forma idiomática en SQL profesional.

03

Vistas

Consultas con nombre · abstracción y seguridad

¿Qué es una vista?

DEFINICIÓN

Una **vista** es una relación virtual: una consulta `SELECT` a la que se le pone nombre. *No* guarda datos: el motor guarda **la definición**. Cada vez que alguien la consulta, el motor sustituye el nombre por la consulta original.

PARA QUÉ SIRVE · SEGURIDAD

Exponer solo un subconjunto de columnas o filas. Ejemplo: que el personal administrativo vea datos de docentes sin el salario.

PARA QUÉ SIRVE · SIMPLICIDAD

Encapsular consultas complejas detrás de un nombre fácil. Quien la usa, no necesita conocer los JOINs ni los filtros.

✓ CONSECUENCIA IMPORTANTE

Como la vista no almacena datos, siempre devuelve información **actualizada**. Si cambia una tabla base, la próxima consulta a la vista lo refleja al instante.

CREATE VIEW y uso

```sql
-- Vista: datos de docentes sin información salarial
CREATE VIEW docentes_publico AS
SELECT id_docente, nombre, apellido, email, id_departamento
FROM docentes;

-- Vista con JOIN: comisiones 2025 con nombre de asignatura y docente
CREATE VIEW comisiones_2025 AS
SELECT c.id_comision, asg.nombre AS asignatura,
       d.apellido AS docente, c.cuatrimestre, c.cupo
FROM comisiones c
INNER JOIN asignaturas asg ON c.id_asignatura = asg.id_asignatura
INNER JOIN docentes d      ON c.id_docente = d.id_docente
WHERE c.anio = 2025;

-- Uso: se consulta como cualquier tabla
```

✓ TIP

Si la vista tiene expresiones o agregados sin alias, se pueden renombrar al definirla: `CREATE VIEW v(col1, col2) AS SELECT ...`. Las vistas también pueden definirse sobre otras vistas.

Vistas materializadas — concepto

VISTA COMÚN

Solo guarda la **definición**. Cada consulta vuelve a ejecutarla. Lectura puede ser lenta si la consulta es pesada.

VISTA MATERIALIZADA

Guarda el **resultado físicamente**. El motor lo mantiene sincronizado cuando cambian las tablas base. Lectura muy rápida, pero hay costo de almacenamiento y de mantenimiento.

Estrategias de refresco:

- **Inmediato** — se recalcula cada vez que cambia una tabla base.
- **Perezoso** — se recalcula cuando alguien la consulta.
- **Periódico** — se recalcula cada X tiempo; entre refrescos, puede estar desactualizada.

⚠ **MYSQL**

MySQL **no tiene vistas materializadas nativas**. Se simulan con tablas comunes + triggers, o con tareas programadas que las refresquen. Oracle y PostgreSQL sí las soportan.

Actualizar a través de una vista

Se puede hacer `INSERT` / `UPDATE` / `DELETE` sobre una vista **si es "actualizable"**. El motor traduce la operación a la tabla base subyacente.

REGLAS DE UNA VISTA ACTUALIZABLE

- El `FROM` referencia una **sola tabla**.
- El `SELECT` lista solo columnas (sin expresiones, sin agregados, sin `DISTINCT`).
- No hay `GROUP BY` ni `HAVING`.
- Los atributos que la vista no expone deben admitir `NULL` (o tener `DEFAULT`).

```sql
-- WITH CHECK OPTION: rechaza inserciones/updates que violen el WHERE de la vista
CREATE VIEW comisiones_2025_check AS
SELECT id_comision, id_asignatura, id_docente, anio, cuatrimestre, cupo
FROM comisiones
WHERE anio = 2025
WITH CHECK OPTION;

-- Esto fallará: violaría el WHERE de la vista
INSERT INTO comisiones_2025_check (id_asignatura, id_docente, anio, cuatrimestre, cupo)
VALUES (5, 7, 2024, 1, 30);   -- anio = 2024 no cumple anio = 2025
```

UNIVERSIDAD CATÓLICA DE CÓRDOBA
JESUITAS
70 AÑOS

04

Modificación de datos

`INSERT`, `UPDATE`, `DELETE` · con seguridad

INSERT — agregar filas

```sql
-- Forma corta: el orden de los valores debe coincidir con el de las columnas en la tabla
INSERT INTO asignaturas
VALUES (NULL, 'Diseño de Bases de Datos', 5, 96, 1, 3);

-- Forma explícita: nombrando columnas (la recomendada, sobrevive a cambios de esquema)
INSERT INTO asignaturas (nombre, creditos, carga_horaria, es_obligatoria, id_departamento)
VALUES ('Diseño de Bases de Datos', 5, 96, 1, 3);

-- INSERT ... SELECT: insertar el resultado de una consulta
-- Copiar a una tabla histórica los alumnos egresados
INSERT INTO alumnos_egresados (id_alumno, nombre, apellido, fecha_egreso)
SELECT a.id_alumno, a.nombre, a.apellido, CURRENT_DATE
FROM alumnos a
INNER JOIN alumno_carrera ac ON a.id_alumno = ac.id_alumno
WHERE ac.estado = 'egresada';
```

ATRIBUTOS NO ESPECIFICADOS

Si una columna no aparece en el `INSERT`, toma su valor por defecto o `NULL` (si la columna lo admite). Si es `NOT NULL` sin `DEFAULT`, la inserción falla.

UPDATE — modificar filas existentes

```sql
-- Aumento del 8% a TODOS los docentes (sin WHERE)
UPDATE docentes
SET salario_base = salario_base * 1.08;

-- Aumento solo a los que ganan menos del promedio (subconsulta escalar en WHERE)
UPDATE docentes
SET salario_base = salario_base * 1.10
WHERE salario_base < (SELECT AVG(salario_base) FROM docentes);

-- Modificar varias columnas a la vez
UPDATE alumnos
SET telefono = '351-555-0000',
    email = CONCAT(LOWER(apellido), '@ucc.edu.ar')
WHERE id_alumno = 42;
```

✓ CÓMO SE PROCESA EL UPDATE CON SUBCONSULTA

El motor evalúa **toda** la subconsulta primero (calcula el promedio actual), y recién después aplica el `UPDATE`. Si lo hiciera fila por fila, el promedio iría cambiando y el resultado dependería del orden interno.

UPDATE con CASE — la trampa del orden

Queremos dar 5 % de aumento a los docentes que cobran menos de 100 000, y 3 % al resto. Pensar bien antes de escribir.

DOS UPDATES EN MAL ORDEN

```sql
UPDATE docentes
SET salario_base = salario_base * 1.05
WHERE salario_base < 100000;

UPDATE docentes
SET salario_base = salario_base * 1.03
WHERE salario_base ⩾ 100000;
```

Quien cobraba 99 000 → sube a 103 950 (con el 5%) → después recibe el 3 % otra vez → termina con ~8 % de aumento.

UNA SOLA SENTENCIA CON CASE

```sql
UPDATE docentes
SET salario_base = CASE
    WHEN salario_base < 100000
        THEN salario_base * 1.05
    ELSE salario_base * 1.03
END;
```

Cada fila se evalúa **una sola vez**: para cada docente se decide qué rama aplicar.

✓ FORMA GENERAL DE CASE

```sql
CASE
  WHEN predicado1 THEN valor1
  WHEN predicado2 THEN valor2
  ELSE valor_por_defecto
END
```

DELETE — borrar filas

```sql
-- Borrar todas las inscripciones de una comisión específica
DELETE FROM inscripcion_alumno_comision
WHERE id_comision = 99;

-- Borrar inscripciones con subconsulta: las de alumnos egresados
DELETE FROM inscripcion_alumno_comision
WHERE id_alumno IN (
    SELECT id_alumno
    FROM alumno_carrera
    WHERE estado = 'egresada'
);

-- Borrar TODAS las filas (peligroso) — pero conserva la estructura
DELETE FROM notas;
```

⚠ LO MÁS PELIGROSO DE SQL

`DELETE FROM tabla;` sin `WHERE` borra TODAS las filas. Si la tabla tiene FK con `ON DELETE CASCADE` en otras tablas, el efecto se propaga. Antes de un `DELETE` sin `WHERE`, asegurate de que sea exactamente lo que querés.

*El motor evalúa la subconsulta entera **antes** de empezar a borrar — igual que en `UPDATE`.*

Tres reglas para no romper datos

Cualquier `UPDATE` o `DELETE` en una base con datos reales debería pasar por estos tres pasos.

1

Probar con SELECT primero

Antes del `DELETE` o `UPDATE`, escribí un `SELECT` con **la misma cláusula WHERE**. Si lo que devuelve no es lo que esperás, no lo ejecutes todavía.

2

Siempre poner WHERE

Si el plan es modificar todo a propósito, escribilo **explícitamente** (un comentario, una condición trivial). El `WHERE` ausente es la causa principal de desastres en producción.

3

Usar transacción

Envolver en `START TRANSACTION; ... COMMIT;` (o `ROLLBACK` si algo salió mal). Lo vemos a fondo al final del cuatrimestre, pero ya es buena costumbre.

```sql
-- Flujo típico
START TRANSACTION;
SELECT * FROM alumnos WHERE fecha_ingreso < '2010-01-01';   -- preview
DELETE FROM alumnos WHERE fecha_ingreso < '2010-01-01';        -- ejecutar
-- Si todo bien:     COMMIT;
-- Si algo salió mal:  ROLLBACK;
```

Pausa: probalo vos

Consigna: aumentar un 15 % el presupuesto de los departamentos que tienen *más de 5 docentes activos*. Mostrar primero el listado con un `SELECT` para verificar.

```sql
-- Paso 1: ver qué departamentos serían afectados
SELECT dep.id_departamento, dep.nombre, dep.presupuesto,
       COUNT(d.id_docente) AS cant_docentes
FROM departamentos dep
LEFT JOIN docentes d ON dep.id_departamento = d.id_departamento
GROUP BY dep.id_departamento, dep.nombre, dep.presupuesto
HAVING COUNT(d.id_docente) > 5;

-- Paso 2: aplicar el aumento (idealmente dentro de una transacción)
START TRANSACTION;

UPDATE departamentos
SET presupuesto = presupuesto * 1.15
WHERE id_departamento IN (
    SELECT id_departamento FROM (
```

¿POR QUÉ LA SUBCONSULTA ENVUELTA EN OTRA?

MySQL no permite hacer `UPDATE` sobre una tabla y referenciarla en una subconsulta directa del `WHERE`. Envolverla en otra subconsulta (tabla derivada) es el workaround estándar.

Lo que vimos hoy

- **Subconsultas en WHERE**: `IN`, `NOT IN`, `EXISTS`, `NOT EXISTS`.
- **EXISTS vs IN**: correlacionada vs no correlacionada; `NOT EXISTS` es robusto ante NULL.
- **SOME / ALL**: mención (preferimos `IN` / `EXISTS`).
- **Otras ubicaciones**: escalares, en `FROM` (tablas derivadas), `WITH` (CTE).
- **Vistas**: `CREATE VIEW`, abstracción y seguridad.
- **Vistas materializadas**: concepto; MySQL no las tiene nativas.
- **Reglas de vista actualizable** + `WITH CHECK OPTION`.
- **DML con subconsultas**: `INSERT ... SELECT`, `UPDATE/DELETE` + `WHERE ... IN (...)`.
- **3 reglas**: `SELECT` primero, `WHERE` siempre, transacción.

PRÓXIMA CLASE · INTEGRIDAD, TRIGGERS, PROCEDURES Y AUTORIZACIÓN

`CHECK`, `ON DELETE CASCADE`, triggers `BEFORE` / `AFTER`, stored procedures (a nivel conceptual), `GRANT` / `REVOKE`.

Práctico de la semana: subconsultas + vistas + DML con seguridad sobre SiGUE.